CRAB: A NOVEL CLUSTERING SCORE USING CLUSTERING WITH RIVALS AND BUDDIES FOR UNSUPERVISED LEARNING

A Thesis

presented to

the Faculty of California Polytechnic State University,

San Luis Obispo

In Partial Fulfillment

of the Requirements for the Degree

Master of Science in Statistics

by

Allen Choi

June 2026

© 2026

Allen Choi

ALL RIGHTS RESERVED

COMMITTEE MEMBERSHIP

TITLE:	CRAB: A Novel Clustering Score using Clustering with Rivals and Buddies for Unsupervised Learning
AUTHOR:	Allen Choi
DATE SUBMITTED:	June 2026

COMMITTEE CHAIR:	Kelly Bodwin, Ph.D. Thesis Advisor
COMMITTEE MEMBER:	Alexander Dekhtyar, Ph.D. Professor of Computer Science and Software Engineering
COMMITTEE MEMBER:	Kevin Ross, Ph.D. Professor of Statistics

ABSTRACT

CRAB: A Novel Clustering Score using Clustering with Rivals and Buddies for Unsupervised Learning

Allen Choi

We present a novel clustering score approach relying on a method that classifies pairs of points as either ”rivals” or ”buddies”, constructing a score for a given number of clusters. We then present results demonstrating the properties of this metric on datasets with no clusters.

Keywords: Clustering. Unsupervised Clustering. Machine Learning.

ACKNOWLEDGMENTS

I'd like to first thank my amazing Mom and Dad for providing me so much emotional support over these past half a decade at Cal Poly. Without them, I don't know how I would have gotten through college.

But without a doubt, in regards to my statistical knowledge and skills I have no one else to thank but the amazing Statistics faculty at Cal Poly, who have always been patient, understanding, and kind.

Also a thank you to Andrew Kerr for providing a quarto template: https://github.com/Wander03/CalPolySTAT-thesis-template.

TABLE OF CONTENTS

Page

LIST OF TABLES vii

LIST OF FIGURES viii

CHAPTER

1 Introduction 1

1.1 Motivation 1

1.1.1 Elbow Method 2

1.1.2 Silhouette Score 4

1.2 Overview of k-Means Clustering 5

1.2.1 k-Means Clustering Details 6

1.3 Literature Review 6

2 Details 9

2.1 Calculated 10

2.2 Sensitivity 10

2.3 Known Data 10

2.4 Performance on penguins dataset 10

3 Computational Details 13

3.1 Matrix Stacking 13

3.2 Algorithm Construction 13

3.3 Score Construction 14

3.4 Point Contribution 17

3.5 Speed and Memory Scaling 17

3.5.1 Scaling with the Number of Observations 17

4 Null Distribution 22

4.1 Null Distribution of Scores 22

4.2 Null Distribution over Resamples 23
4.3 Adj-CRAB Score 23
5 Conclusion 25
REFERENCES 26
APPENDICES
A Resample Matrix 29
B Mean Matrix 33
C Metric Calculation 34
D Point Contribution Function 35
E Get Point Vector Helper 36
F CRAB Score Across k Clusters 37

LIST OF TABLES

Table Page

4.1 Variances for 20% Resample Calculation 23

LIST OF FIGURES

Figure		Page
1.1	Scatterplot of the Example Dataset	3
1.2	WCSS Decreases as Number of Clusters k Increases	3
1.3	Silhouette Score as Number of Clusters k Increases	5
1.4	Three 60% Resamples of the iris dataset for Petal Length vs. Petal Width	7
1.5	Three 60% Resamples of a 2-D Random Normal Distribution	8
3.1	Plot of Penguin Species by Bill and Flipper Length	14
3.2	The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k = 2	15
3.3	The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k = 3	15
3.4	Speed of the CRAB Algorithm Quadratically Increases as Number of Observations Increases	18
3.5	Memory of the CRAB Algorithm Quadratically Increases as Number of Observations Increases	19
3.6	Speed of the CRAB Algorithm Increasing Linearly as Number of Dimensions Increases	20
3.7	Memory of the CRAB Algorithm Increasing Linearly as Number of Dimensions Increases	21
4.1	Distribution of Scores on a Random p-Dimensional Normal Distribution, Faceted by Resample %	22

Chapter 1

INTRODUCTION

1.1 Motivation

Unsupervised clustering algorithms are commonly used today when attempting to classify observations in groups where the observation's membership or the number of groups is not known beforehand. Clustering is useful when attempting to get an overview of groups across a dataset, which provides insight into which data points are most similar to other data points. This can provide implicit signals into common traits shared by observations in the same group, which is valuable across a wide range of fields such as biology (Nugent & Meila, 2010), engineering (Ababneh, Khodier, & Khatalin, 2025), health education (Roper, He, Perez-Concha, & Guthridge, 2023), physiology (Bird et al., 2022), and industry (John, Shobayo, & Ogunleye, 2023).

One of the main difficulties associated with unsupervised clustering is due to the fact that the number of clusters or groupings in the dataset is not known beforehand. This can be circumvented through the use of domain knowledge, but even in situations with no prior knowledge there still exist methods to estimate the optimal number of groups. Methods such as the elbow method and Silhouette score (Kodinariya & Makwana, 2013) are commonly used to compare multiple candidate values of k over a range to obtain a specific "best-guess" of this optimal value. However, even with these methods there are often disagreements and caveats that we will further explore. In this thesis we introduce a novel method for determining the optimal number of clusters using a resampling-based approach over many subsamples of a dataset.

1.1.1 Elbow Method

The elbow method (Thorndike, 1953) is a relatively simple technique that utilizes the plotting of the sum of the Within Cluster Sum of Squares (WCSS) across all clusters at the y axis, against the number of clusters on the x-axis. Adding more clusters necessarily decreases the WCSS due to the fact that adding an extra cluster will mean that at least one observation is closer to that cluster than the others. The question of "how many clusters" is then answered by looking at where this drop-off in WCSS is significant, as seen in the example dataset below. This method however comes with many problems (Schubert, 2023). The biggest issue is that while for some datasets this "drop-off" in WCSS is easy to visually recognize, sometimes it can be extremely difficult to tell what is considered significant. In fact, Schubert takes a strong approach on this, where they state "Educators should omit the method or at least explain better alternatives".

We demonstrate this below with an example dataset, where:

$$x_1, y_1 \sim gamma(\alpha = 1)$$
$$x_2, y_2 \sim gamma(\alpha = 1.5) + 2$$
$$x_3, y_3 \sim gamma(\alpha = 2) + 4$$

Fifty observations were randomly generated from each distribution, for a total of 150 observations (X, Y) across the three distributions.

An overview of this data can be seen below, where the observations are more obviously separated into three clusters, as taken from their respective distributions.

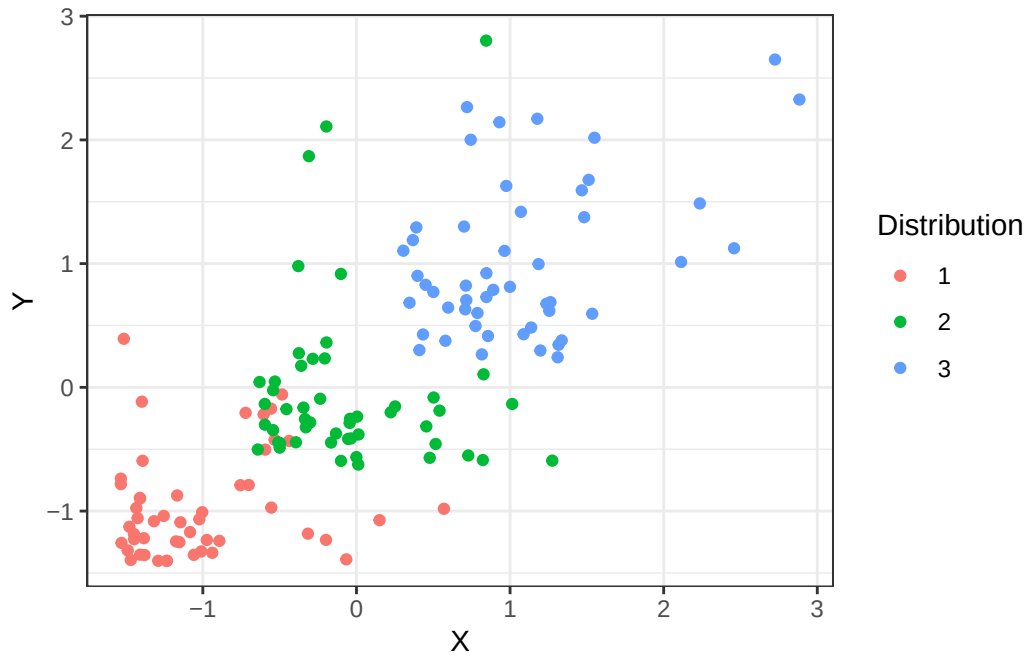


Figure 1.1: Scatterplot of the Example Dataset

Using this data, we used k-Means clustering across a set of candidate k cluster values from one to ten, visualizing the WCSS across all the k values.

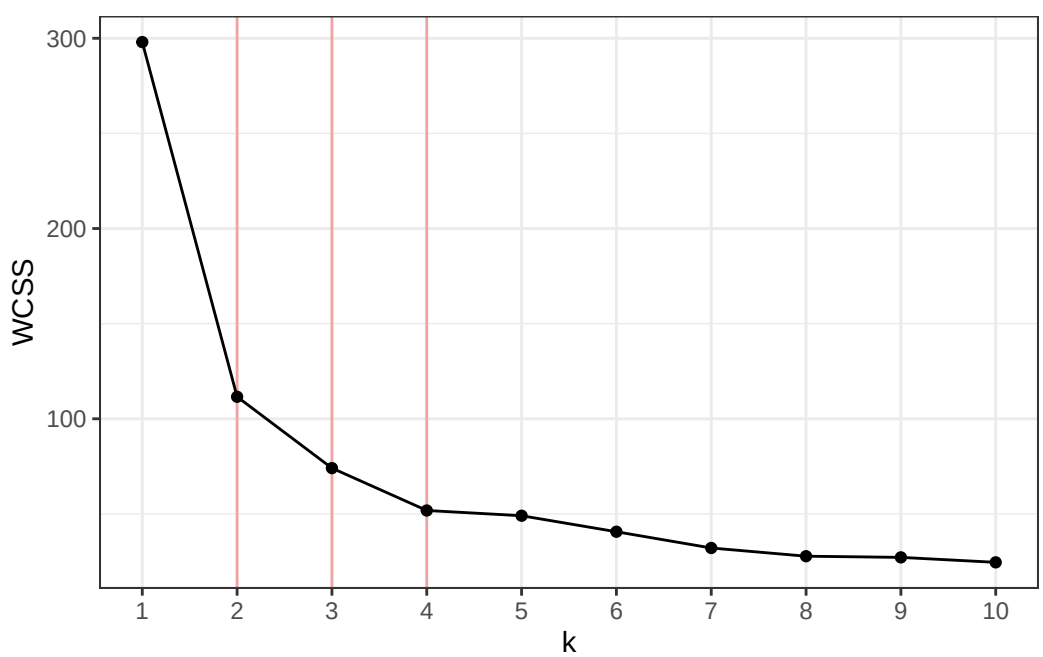


Figure 1.2: WCSS Decreases as Number of Clusters k Increases

As seen in the plot above, there exists no obvious "elbow" at which point the decrease in WCSS changes dramatically; any choice of k between 2 and 4 is a justifiable answer, which makes the ideal choice of k ambiguous.

1.1.2 Silhouette Score

The Silhouette Score approach (Rousseeuw, 1987) is a method proposed to measure how comfortably each observation belongs to its assigned cluster compared to the other possible clusters. As it is defined in the original 1986 paper, the silhouette width $s(i)$ for observation i is defined by:

$$s(i) = \frac{b(i) - a(i)}{max\{a(i), b(i)\}}$$

where $a(i)$ is defined by the average distance between the i and all other observations in the same cluster, and $b(i)$ is defined by the smallest average distance between i and all other observations in each other respective clusters.

Averaging this score gives a measure of how well a certain number of clusters fits the data, similar to the elbow method. Unlike in the elbow method however, where increasing the number of clusters necessarily decreases the WCSS, the average Silhouette width does not necessarily increase as the number of clusters increases. A high Silhouette Score for a given k means that k is a good number of clusters for that given dataset.

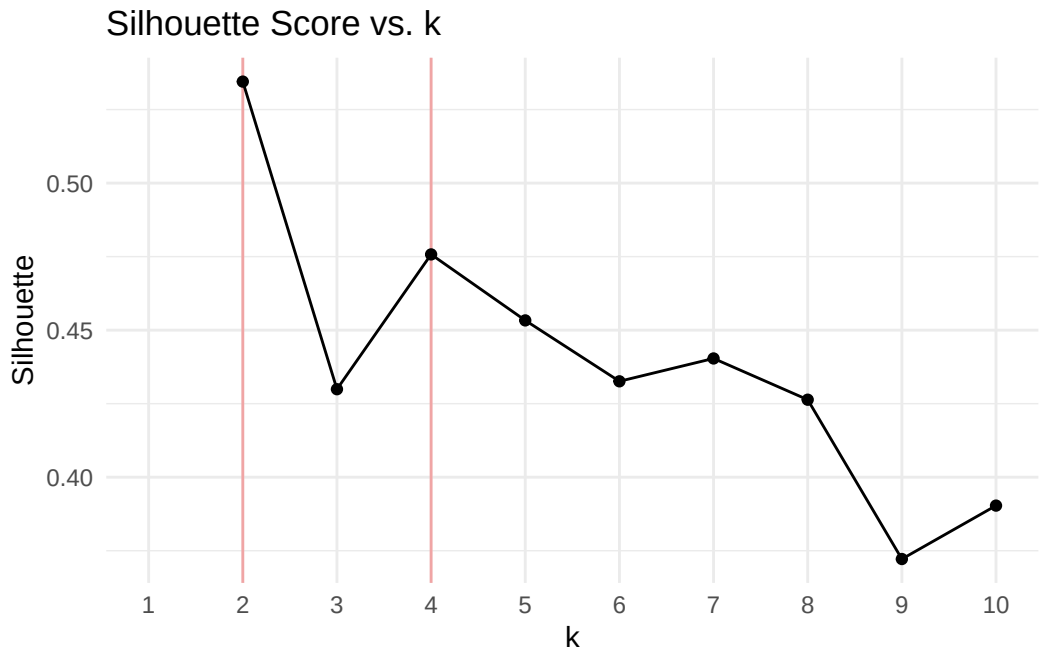


Figure 1.3: Silhouette Score as Number of Clusters k Increases

When looking at the Silhouette Scores on the same example dataset where the elbow method was used, we see that the Silhouette Score are highest for $k = 2$ and next-highest for $k = 4$, indicating that the optimal number of clusters is 2 or 4 for this dataset. Since we know beforehand that the data is distributed from three gamma distributions, this answer is not "correct".

1.2 Overview of k-Means Clustering

For the scope of this paper we will go over only the k-Means clustering algorithm (MacQueen, 1967). However, in theory the CRAB algorithm framework allows it to work on any unsupervised method as long as the algorithm outputs a cluster assignment for every observation in the dataset. Alternatives such as hierarchical agglomerative clustering and mixture models (Jaeger & Banks, 2023) would also work using this framework.

1.2.1 k-Means Clustering Details

In terms of unsupervised clustering, k-Means clustering is one is the most well-known as well as simplest methods available. In essence, the algorithm seeks to find clustering assignments that minimize the average squared distance between a set of points and various centroids k that are randomly initialized in a dataset. The clustering results are highly dependent on the initial starting positions of the centroids due to the fact that k-Means often converges to local minima, and when the usual Euclidean distance metric is used it often finds spherical clusters that are centered around these centroids (Ikotun, Ezugwu, Abualigah, Abuhaija, & Heming, 2023). The speed and memory complexity of the algorithm will be explored in Section 3.5.

1.3 Literature Review

The main mechanism the CRAB algorithm attempts to leverage is the fact that on a given dataset, if there is an obvious clustering assignment present for the observations, then that overall clustering should also be present in a random subsample of the dataset. Barring very unusual and unlucky resamples, how many clusters there should be and where those clusters are roughly located should stay consistent across resamples.

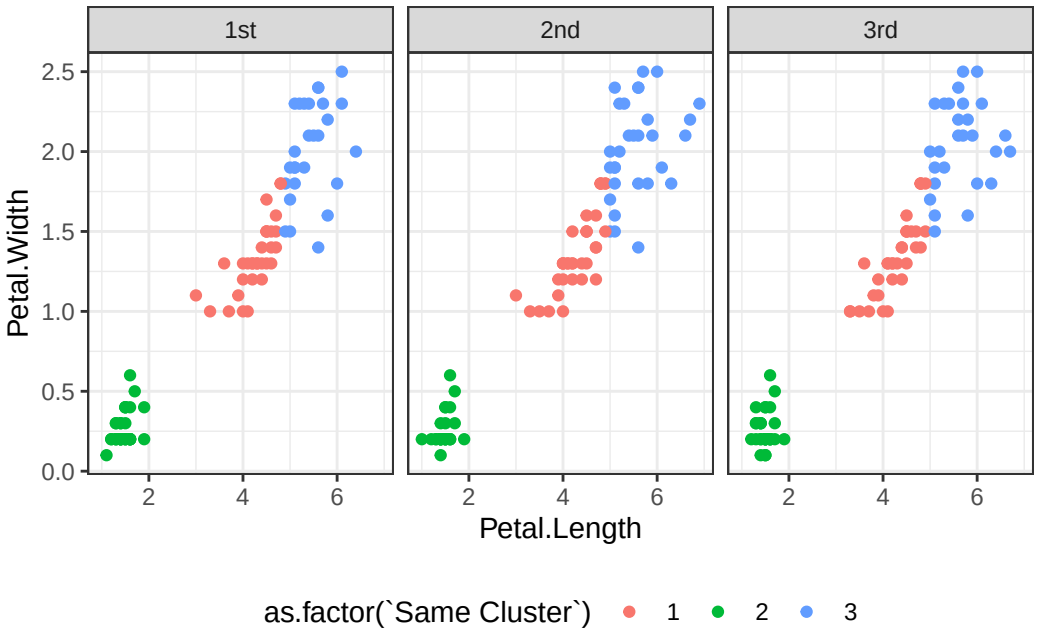


Figure 1.4: Three 60% Resamples of the iris dataset for Petal Length vs. Petal Width

This concept is demonstrated here, where even with random resamples, the general location of the clusters are similar. More importantly, though, is the fact that observations that are in the same cluster tend to stay in the same cluster, given that there is a clear clustering structure observable.

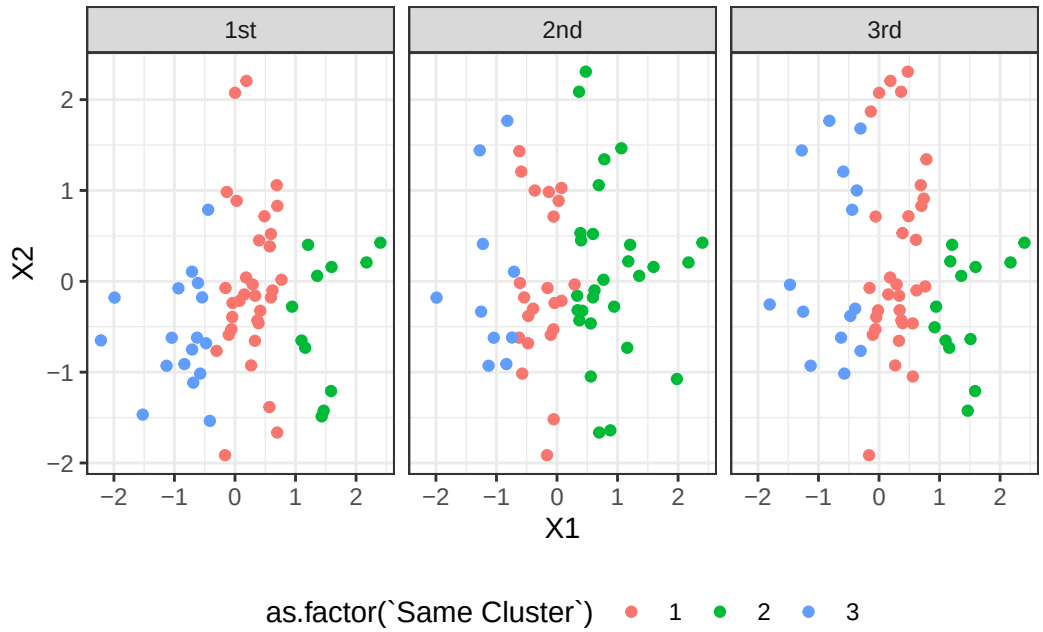


Figure 1.5: Three 60% Resamples of a 2-D Random Normal Distribution

As seen in the plot above, when the clustering structure is not clearly identifiable through k-Means, the clustering assignments outputted from k-Means tends to be significantly different from resample to resample. This is in contrast to the iris dataset example above with three 60% resamples; the clustering assignments tend to be similar even after the resampling procedure was completed.

Much previous work has been done in leveraging resamples in order to inform the number of clusters. One paper computes clustering assignments completed on random resamples on the full dataset, then comparing those resampled cluster assignments against the clustering assignment completed on the full dataset (Levine & Domany, 2001). Another paper obtains the clustering assignments on two random subsamples of a dataset, then computes a metric on how similar the assignments are between those two resamples (Ben-Hur, Elisseeff, & Guyon, 2001). We seek to extend this concept by running a large number of resamples on random resamples of a dataset, then combining the resample matrices obtained from each respective cluster assignments into a score on a cluster-by-cluster basis.

Chapter 2

DETAILS

Given two distinct points and their cluster assignments across resamples, the clustering assignments should be consistent if there is an obvious clustering pattern in the dataset. For example, two points that are far away from each other should consistently not be in the same cluster, and two points that are close together should consistently be in the same cluster, even across resamples. In k-Means, this means that there should be spherical clusters that are relatively far apart for the cluster assignments to be consistent across resamples; a large overlap between groups means that after even after a resample, a large proportion of observations will be not in the same cluster or "enemies" with their neighboring observations, and vice versa.

The CRAB algorithm in detail is defined below:

1. Given two distinct points and their cluster assignments across resamples, the clustering assignments should be consistent if there is an obvious clustering pattern in the dataset.
2. Given two distinct points and their cluster assignments across resamples, the clustering assignments should be consistent if there is an obvious clustering pattern in the dataset.
3. Given two distinct points and their cluster assignments across resamples, the clustering assignments should be consistent if there is an obvious clustering pattern in the dataset.
4. Given two distinct points and their cluster assignments across resamples, the clustering assignments should be consistent if there is an obvious clustering pattern in the dataset.

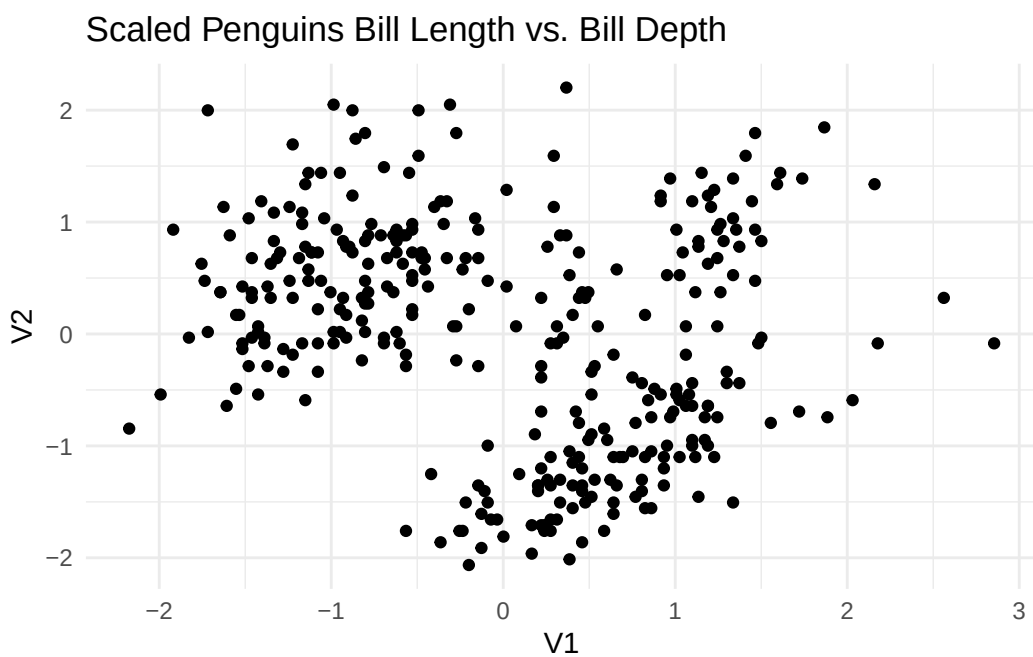
5. Given two distinct points and their cluster assignments across resamples, the clustering assignments should be consistent if there is an obvious clustering pattern in the dataset.

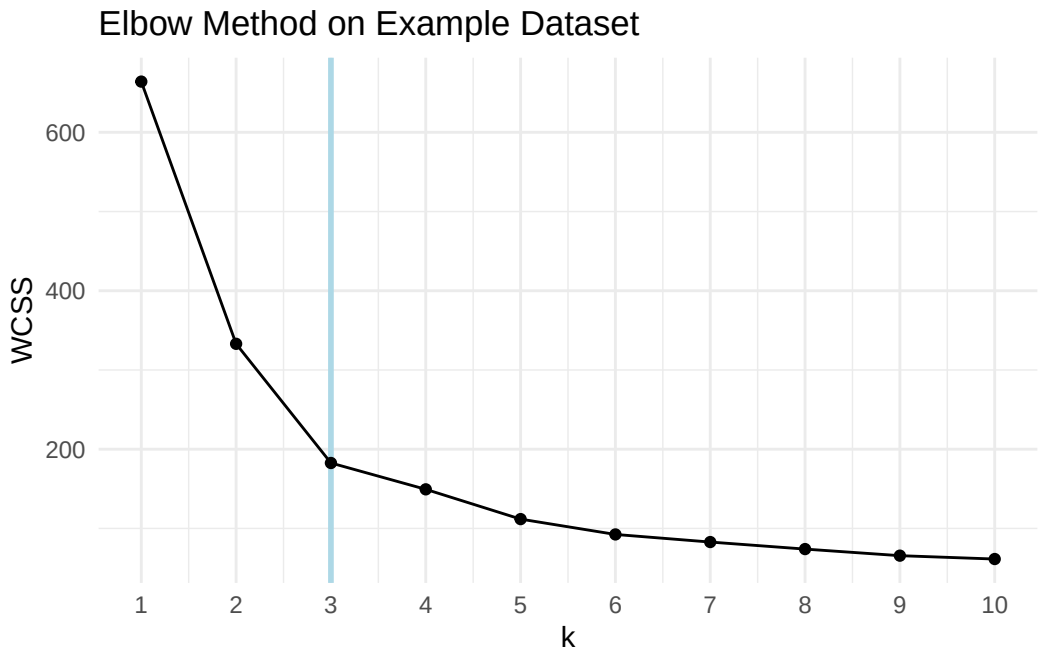
2.1 Calculated

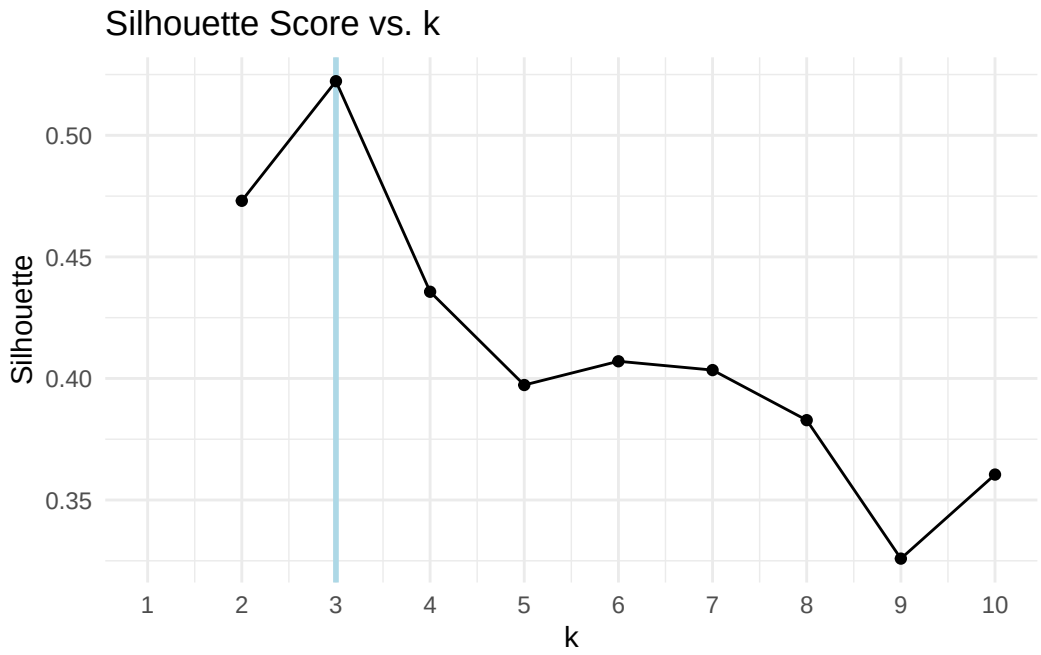
2.2 Sensitivity

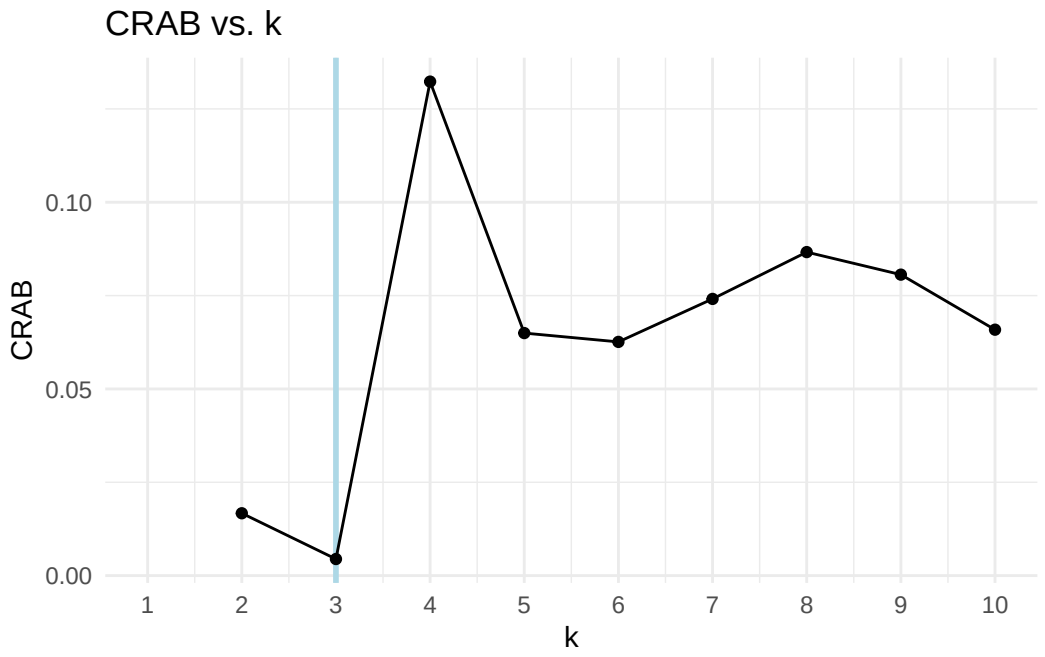
2.3 Known Data

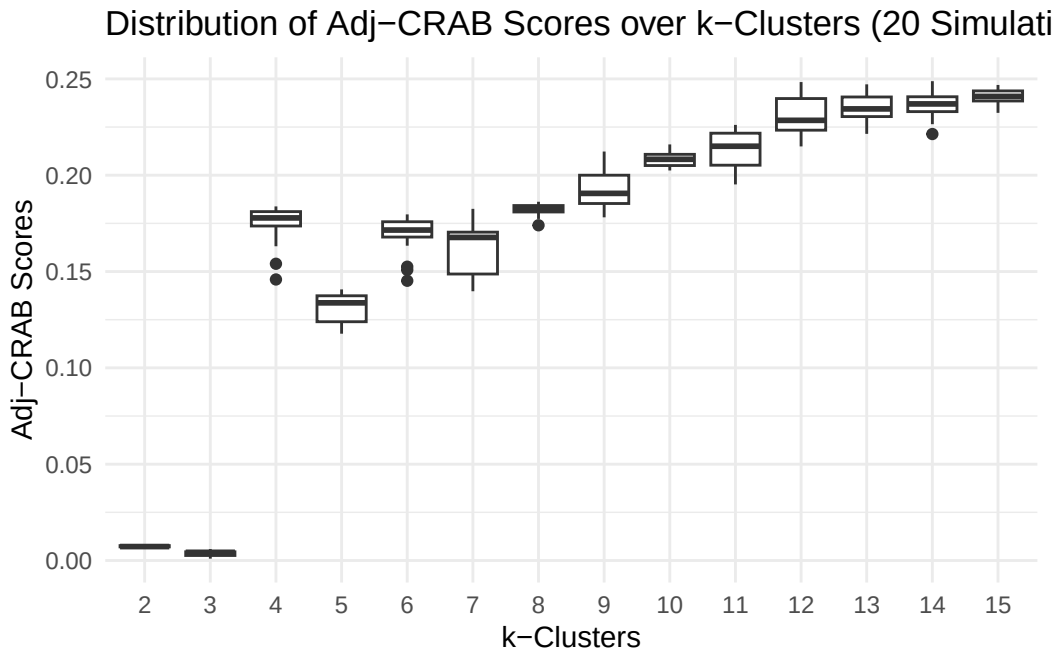
2.4 Performance on penguins dataset




Elbow Method on Example Dataset
WCSS
600
400
200
1
2
3
4
5
6
7
8
9
10
k


Silhouette Score vs. k
Silhouette
0.50
0.45
0.40
0.35
1
2
3
4
5
6
7
8
9
10
k


CRAB vs. k
CRAB
0.00
0.05
0.10
1
2
3
4
5
6
7
8
9
10
k


Distribution of Adj−CRAB Scores over k−Clusters (20 Simulati
Adj−CRAB Scores
0.00
0.05
0.10
0.15
0.20
0.25
2
3
4
5
6
7
8
9
10
11
12
13
14
15
k−Clusters

Chapter 3

COMPUTATIONAL DETAILS

3.1 Matrix Stacking

We now need a framework for the handling of resampling the matrices.

We define a "resample matrix" or "cluster assignment matrix" for each resample, where for an $n \times n$ resample matrix m, for every pairwise interaction between observation i and observation j we have a value of 1 in the matrix if they belong to the sample cluster, -1 if not, and 0 if either point was not in the resample.

Denoted another way, for two points i, j in the resample matrix, the element p_{ij} of the resample matrix is defined as:

$$p_{ij} = \begin{cases} 1, & \text{if points i, j belong to the same cluster} \\ -1, & \text{if points i, j do not belong to the same cluster} \\ 0, & \text{if either point i or point j are not in the resample} \\ \text{NA}, & \text{if pairwise element ij is located in the lower triangle or diagonal} \end{cases}$$

3.2 Algorithm Construction

After we have m matrices, we then reduce the matrices, taking the mean with respect to each index pair, ignoring zeroes. We then receive a final "mean matrix" of size n x n, where n is the number of observations in the dataset.

3.3 Score Construction

To illustrate the construction of the final score given this mean matrix, we use the penguins dataset in R with three known species and scaled bill length and flipper lengths.

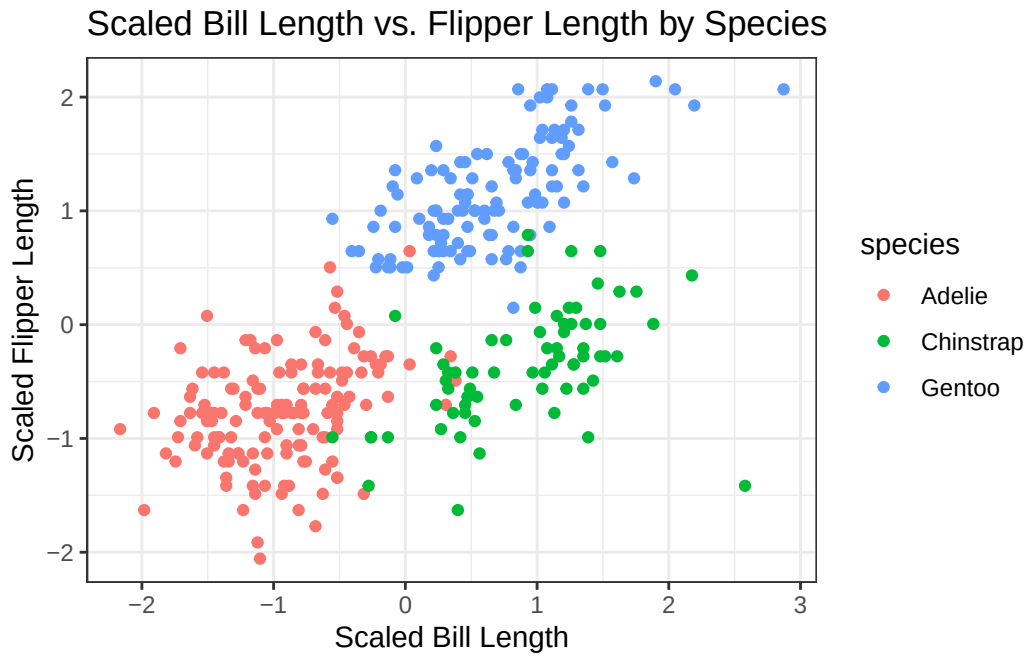


Figure 3.1: Plot of Penguin Species by Bill and Flipper Length

For the purposes of designing the scoring algorithm, we used this dataset with the three "known" clusters in order to fine-tune the behavior of the scores. Ideally, a scoring algorithm would output a lower score when the number of clusters input into the algorithm is not equal to three. Before defining this algorithm, it was helpful to first see the raw mean vector of values after the mean_matrix function, which simply averages the p_{ij} values over all the resample matrices. Note that the values of ones and negative ones were removed, since a vast majority of points either consistently are in the same cluster or are consistently not in the same cluster.

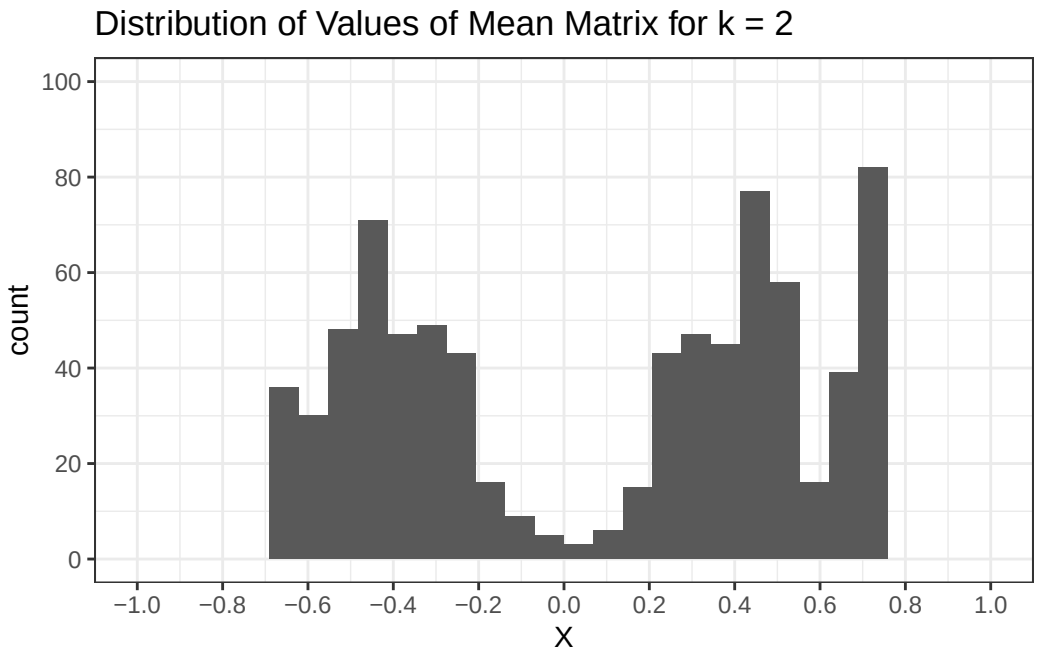


Figure 3.2: The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k = 2

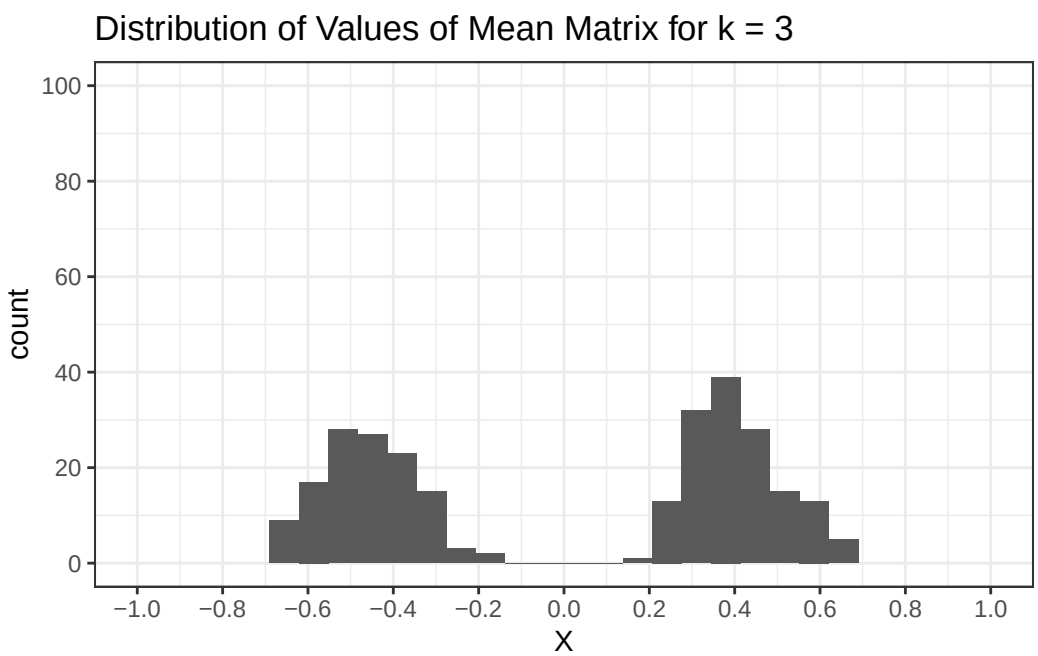


Figure 3.3: The distribution of values inside the mean matrix after running the algorithm on the penguins dataset, using K-Means and k = 3

We see that from the above figures that the distribution of values inside the mean matrix

changes depending on the number of clusters set beforehand in the K-Means algorithm, albeit slightly. For k = 2, there are a small number of points around the value of 0, which indicates that these specific points are ones who are changing "allegiances" often, meaning that they switch from being "buddies" (being in the same cluster) and to being "rivals" (not being in the same cluster).

This informed our next direction of how to construct the scoring algorithm combining all of this information: Our algorithm should heavily penalize points that constantly change allegiances and penalize points that are consistent (whether consistently rivals or consistently buddies) less. Pairs of observations that are consistently in the same cluster would have an average p_{ij} value of 1 or close, and pairs of observations that are consistently not in the same cluster would have an average p_{ij} value of -1 or close. Then, pairs of observations that switch allegiances would have p_{ij} values close to 0.

To make a final score taking all this into account, we decided to take the absolute value of the vectorized mean matrix, which as a side effect would give consistent buddies and consistent rivals the same weight in our score. With this, we would obtain the sum of squared distances from 1, allowing us to penalize those pairs of observations that are inconsistent.

Our final scoring algorithm ends up as:

$\sum_i^n \sum_j^n (1 - \bar{p}_{ij})^2$

, where $\bar{p}_{ij}$ is the average of the p_{ij} values over all m resample matrices.

3.4 Point Contribution

3.5 Speed and Memory Scaling

3.5.1 Scaling with the Number of Observations

As mentioned previously in the paper, the CRAB algorithm involves the construction of m resample matrices of size $n \times n$, where n is the number of observations in the dataset. Predictably, the time complexity of the algorithm roughly scales by the order of $m \cdot n^2$, without taking into account the complexity of the resample matrix construction or the clustering algorithm itself. This is because the m $n \times n$ resample matrices are reduced to a single mean matrix for the calculation of the CRAB score.

With the k-Means algorithm, it has a time complexity that scales by the order of $n \cdot p \cdot q \cdot k$, where n is the number of observations, p is the number of dimensions, q is the number of iterations, and k is the number of clusters (Wang & Song, 2011). For our purposes we will only focus on the number of observations and number of dimensions due to the fact that the maximum number of iterations is capped at 10 and the number of clusters tested with the CRAB algorithm is small (less than 40). The k-Means algorithm is ran for each resample matrix as well.

Taking these time complexities into account, we can expect the entire algorithm to roughly scale at $O(m \cdot (n^2 + np))$, where m is the number of resample matrices, n is the number of observations, and p is the number of dimensions of the dataset.

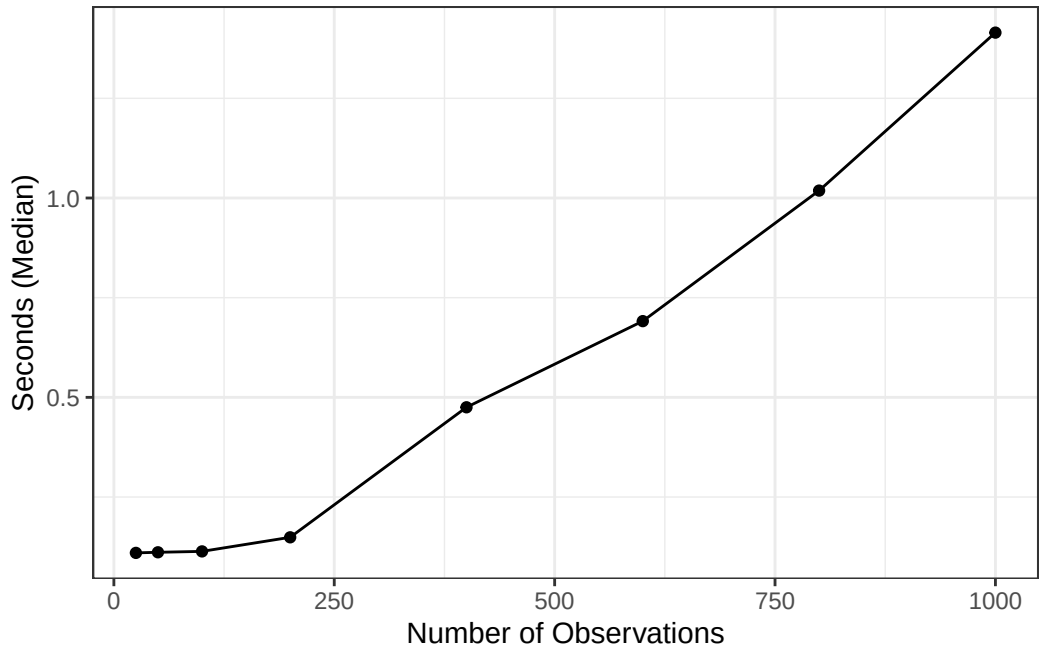


Figure 3.4: Speed of the CRAB Algorithm Quadratically Increases as Number of Observations Increases

As for memory scaling, due to the fact that m total $n \times n$ resample matrices are constructed, we can also expect memory usage to scale quadratically with respect to the number of observations, regardless of the clustering algorithm used. This behavior can be seen when testing the CRAB algorithm with k-Means using the **bench::mark** function. It is important to note that a single score was computed from the results of a single call of the CRAB algorithm on a specific k of three using 50 resamples on a random p-dimensional Normal distribution with n observations; in actuality to compare the viability of each k the CRAB algorithm would need to be run k times, once for each k of interest. The full details and code of the implementation are available on the GitHub repository under **analyses/speed_memory_scaling.qmd**.

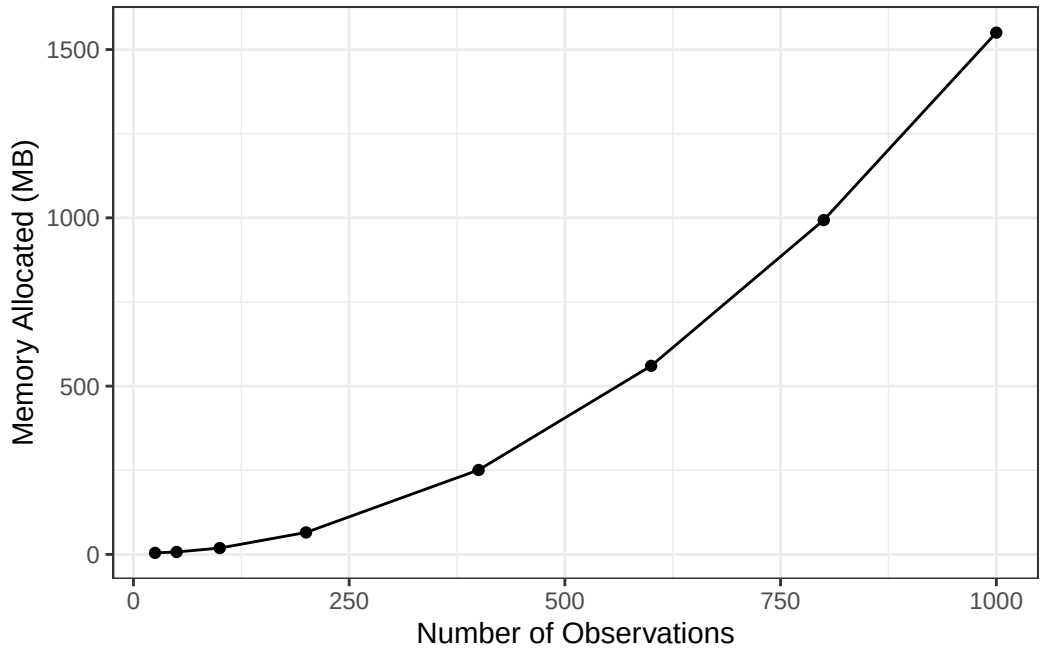


Figure 3.5: Memory of the CRAB Algorithm Quadratically Increases as Number of Observations Increases

This, of course, is not ideal, but does guide possible future work improving the algorithm. At a large n the resample matrices become extremely sparse, leading to possible memory savings if treated as such. However, the fact remains that the algorithm relies on the storage of all m resample matrices in some capacity.

Unlike when changing the number of observations, increasing the number of dimensions should not have a large impact on the speed and memory performance of the CRAB algorithm itself. Since we rely on the clustering assignments output from the clustering algorithm for the CRAB implementation, the actual differences in time and memory usage come almost completely from the clustering algorithm used. As stated earlier, the time complexity of the k-Means algorithm scales linearly, scaling with np.

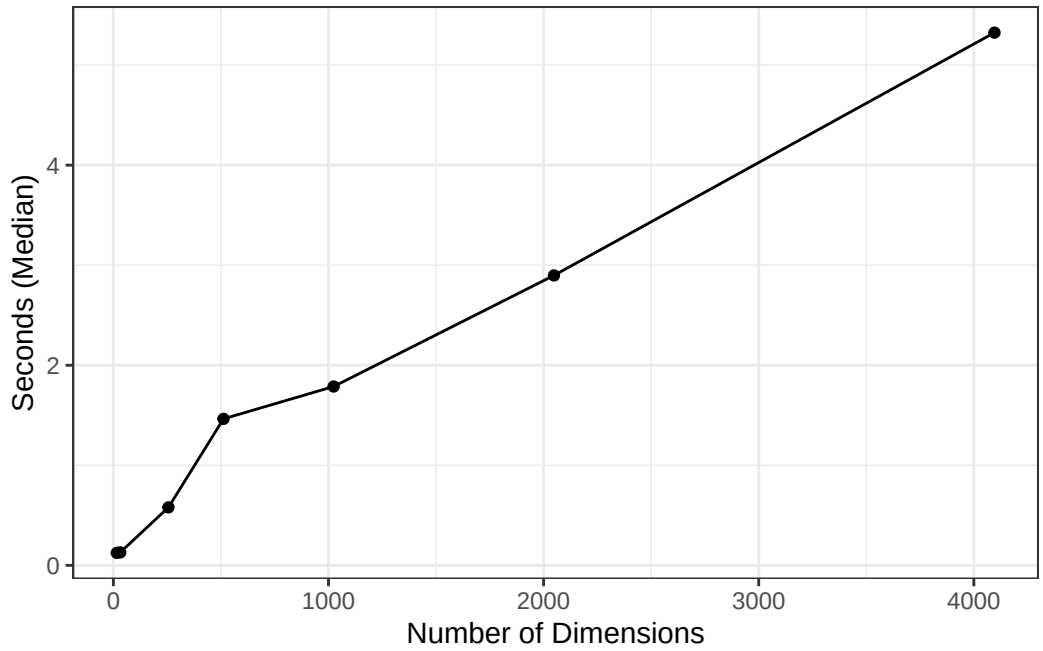


Figure 3.6: Speed of the CRAB Algorithm Increasing Linearly as Number of Dimensions Increases

This is verified through simulation where running the CRAB algorithm on increasing p dimensions only scales the time linearly with respect to p. The k-Means algorithm has a space complexity of $O((n + k) \cdot p)$, where p is the number of dimensions of the dataset (Ikotun et al., 2023). If we hold n and k to be constant, we would also expect the algorithm to scale linearly in terms of time in respect to the number of dimensions p.

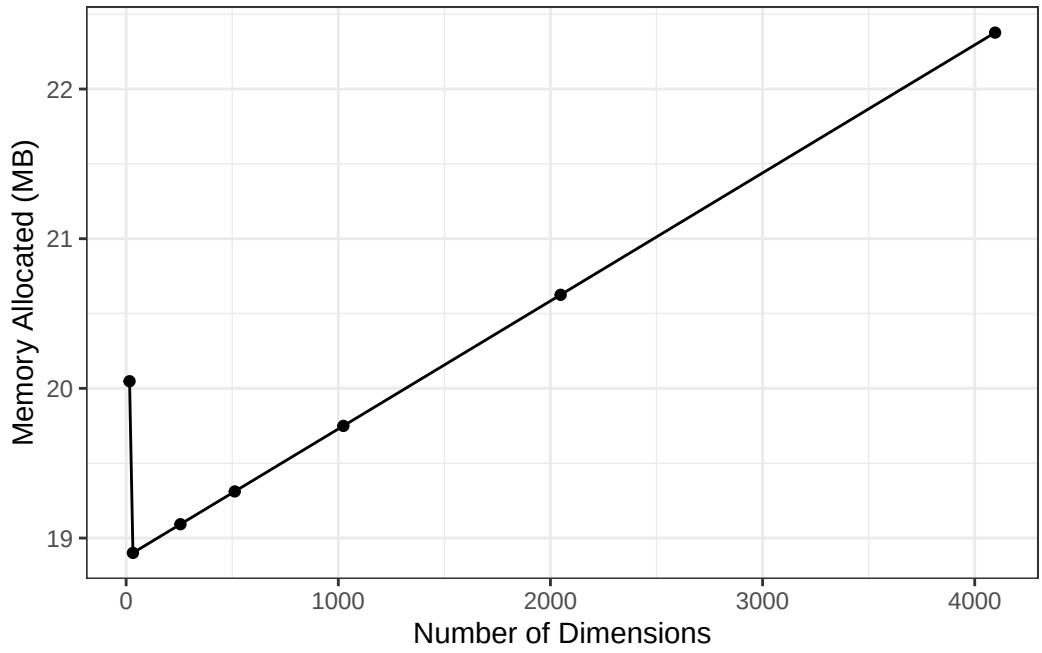


Figure 3.7: Memory of the CRAB Algorithm Increasing Linearly as Number of Dimensions Increases

Through simulation, barring the anomalous behavior with a small number of dimensions, the expected memory usage almost exactly follows this behavior where the memory allocation increases linearly with respect to p dimensions.

Chapter 4

NULL DISTRIBUTION

4.1 Null Distribution of Scores

It is also interesting information to see what the distribution of scores looks like when there are no clusters in the dataset.

The hope was that our scores can be adjusted depending on the behavior of the CRAB scores when there are in actuality no clusters (or in this case one cluster). Based off the behavior of this "null distribution", we can adjust the CRAB score.

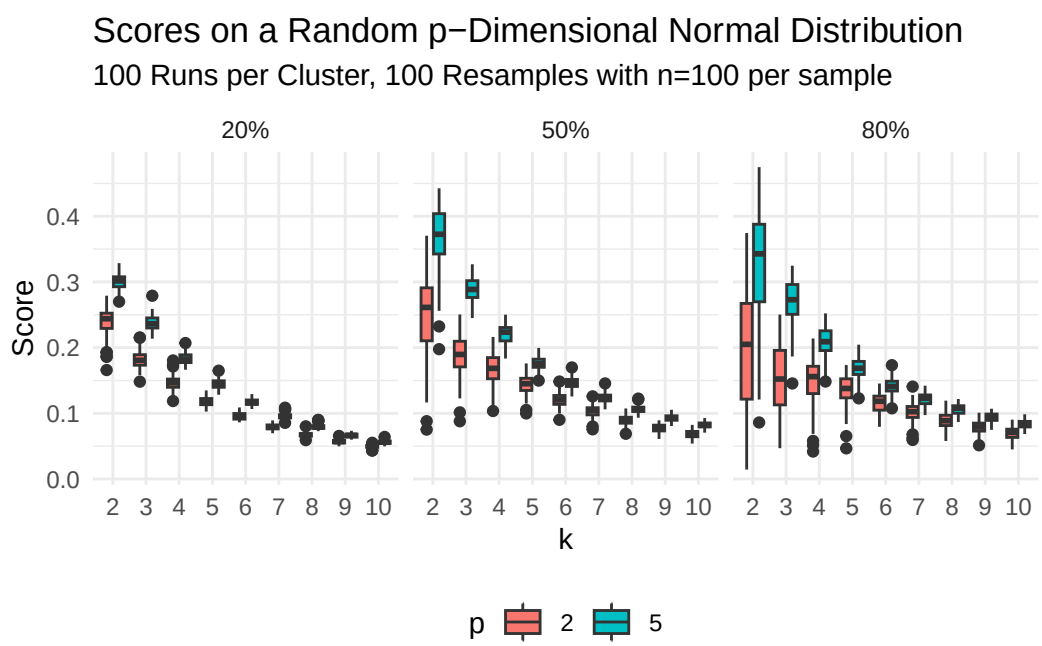


Figure 4.1: Distribution of Scores on a Random p-Dimensional Normal Distribution, Faceted by Resample %

We see in the above figure that the resample proportion has a significant impact on the distribution of scores, where the variability and mean of scores over many simulation runs

is dependent on the resample proportion. The scores also tend to be higher, scaling with p-dimensions, with a decreasing score distribution as k increases. Notably, all these scores look symmetrically distribution around their mean, with respect to each k.

4.2 Null Distribution over Resamples

```
`summarise()` has grouped output by 'factor(k)'. You can override using the
`.groups` argument.
```

Table 4.1: Variances for 20% Resample Calculation

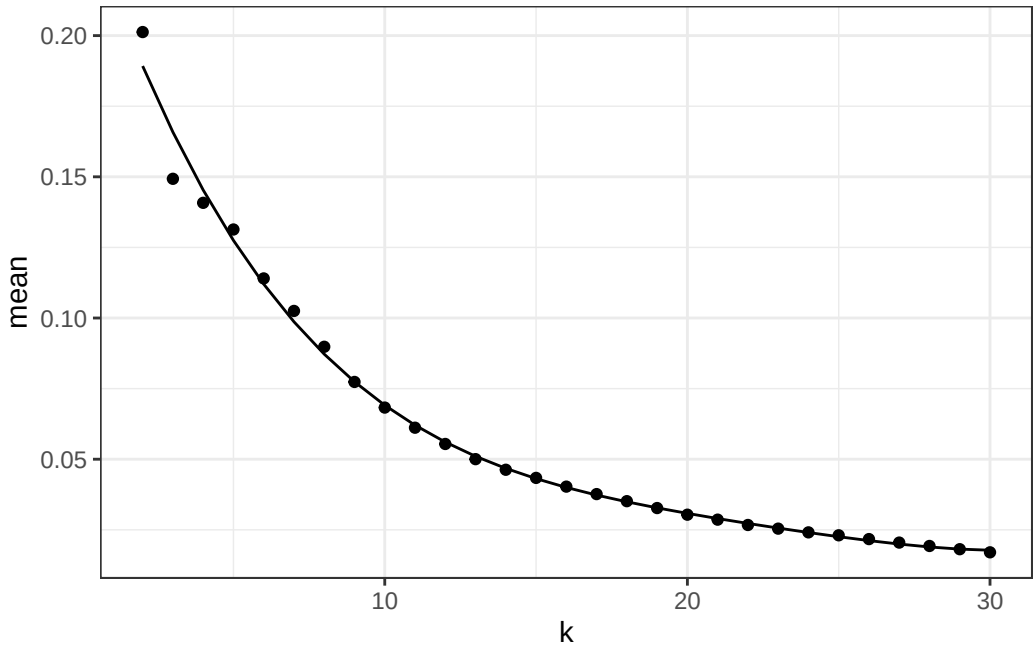
factor(k)	factor(prop_resample)	var
2	0.2	0.1347830
3	0.2	0.1471547
4	0.2	0.1285054
5	0.2	0.1161971
6	0.2	0.1133949

4.3 Adj-CRAB Score

The need to adjust the raw CRAB scores then becomes obvious; if the score through simulation displays monotonically decreasing behavior, then we should adjust any scores that we get as a function of the parameters of interest as necessary (n, resample proportion, k). To find a good approximation of this, we ran 50 simulated 2-dimensional normal distribution datasets, and for each dataset ran the CRAB algorithm from 2 to 30 clusters. The median CRAB score with respect to each cluster k was then taken.

```
1          2          3          4          5          6          7
```

```
0.18922980 0.16581470 0.14532410 0.12747757 0.11200921 0.09866764 0.08721601
         8          9         10         11         12         13         14
0.07743199 0.06910778 0.06205010 0.05608019 0.05103382 0.04676128 0.04312740
        15         16         17         18         19         20         21
0.04001151 0.03730747 0.03492368 0.03278306 0.03082304 0.02899558 0.02726717
        22         23         24         25         26         27         28
0.02561883 0.02404608 0.02255900 0.02118216 0.01995467 0.01893017 0.01817681
        29
0.01777727
```



Chapter 5

CONCLUSION

Note that this means that algorithms such as DBSCAN (Density-Based Spatial Clustering of Applications with Noise) will not work with this framework due to the fact that some observations are classified as “noise” (Ester, Kriegel, Sander, & Xu, 1996).

On the penguins dataset, we use 100 resamples and a resample proportion of 80% wih the k-Means algorithm and a given k, using again the scaled bill length and flipper lengths in our dataset.

(i guess this should also go in the analysis portion??)

REFERENCES

Ababneh, J. I., Khodier, M. M., & Khatalin, S. M. (2025). Energy Optimization in Three-Dimensional Wireless Sensor Networks Using Hybrid K-Means and Mixed Integer Linear Programming. *IEEE Access*, *13*, 163276–163287. https://doi.org/10.1109/ACCESS.2025.3611305

Ben-Hur, A., Elisseeff, A., & Guyon, I. (2001). A stability based method for discovering structure in clustered data. *Biocomputing 2002*. Kauai, Hawaii, USA: WORLD SCIENTIFIC. https://doi.org/10.1142/9789812799623_0002

Bird, M. B., Mi, Q., Koltun, K. J., Lovalekar, M., Martin, B. J., Fain, A., … Nindl, B. C. (2022). Unsupervised Clustering Techniques Identify Movement Strategies in the Countermovement Jump Associated With Musculoskeletal Injury Risk During US Marine Corps Officer Candidates School. *Frontiers in Physiology*, *13*, 868002. https://doi.org/10.3389/fphys.2022.868002

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, 226–231. Portland, Oregon: AAAI Press.

Ikotun, A. M., Ezugwu, A. E., Abualigah, L., Abuhaija, B., & Heming, J. (2023). K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data. *Information Sciences*, *622*, 178–210. https://doi.org/10.1016/j.ins.2022.11.139

Jaeger, A., & Banks, D. (2023). Cluster analysis: A modern statistical review. *WIREs Computational Statistics*, *15*(3). https://doi.org/10.1002/wics.1597

John, J. M., Shobayo, O., & Ogunleye, B. (2023). An Exploration of Clustering Algorithms for Customer Segmentation in the UK Retail Market. *Analytics*, *2*(4), 809–823. https:

//doi.org/10.3390/analytics2040042

Kodinariya, T., & Makwana, P. (2013). Review on Determining of Cluster in K-means Clustering. *International Journal of Advance Research in Computer Science and Management Studies*, *1*, 90–95.

Levine, E., & Domany, E. (2001). Resampling Method for Unsupervised Estimation of Cluster Validity. *Neural Computation*, *13*(11), 2573–2593. https://doi.org/10.1162/089976601753196030

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics* (Vol. 5.1, pp. 281–298). University of California Press. Retrieved from https://projecteuclid.org/ebooks/berkeley-symposium-on-mathematical-statistics-and-probability/Proceedings-of-the-Fifth-Berkeley-Symposium-on-Mathematical-Statistics-and/chapter/Some-methods-for-classification-and-analysis-of-multivariate-observations/bsmsp/1200512992

Nugent, R., & Meila, M. (2010). An overview of clustering applied to molecular biology. *Methods in Molecular Biology*, *620*, 369–404. https://doi.org/10.1007/978-1-60761-580-4_12

Roper, L., He, V. Y., Perez-Concha, O., & Guthridge, S. (2023). Complex early childhood experiences: Characteristics of Northern Territory children across health, education and child protection data. *PloS One*, *18*(1), e0280648. https://doi.org/10.1371/journal.pone.0280648

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, *20*, 53–65. https://doi.org/https://doi.org/10.1016/0377-0427(87)90125-7

Schubert, E. (2023). Stop using the elbow criterion for k-means and how to choose the number of clusters instead. *ACM SIGKDD Explorations Newsletter*, *25*(1), 36–42. https://doi.org/10.1145/3606274.3606278

Thorndike, R. L. (1953). Who belongs in the family? *Psychometrika*, *18*(4), 267–276. https://doi.org/10.1007/BF02289263

Wang, H., & Song, M. (2011). Ckmeans.1d.dp: Optimal k-means Clustering in One Dimension by Dynamic Programming. *The R Journal*, *3*(2), 29–33. Retrieved from https://pmc.ncbi.nlm.nih.gov/articles/PMC5148156/

APPENDICES

Appendix A

RESAMPLE MATRIX

```
resample_function <- function(data = data,
                              formula = ~ .,
                              k = 3,
                              number_of_resamples = 15,
                              proportion_resample = 0.9,
                              starting_seed = 599,
                              algorithm = "kmeans") {
  data <- data |>
    drop_na()
  data$index <- 1:nrow(data)
  results <- list()

  # For loop
  results <- foreach(i = 1:number_of_resamples,
                     .packages = c("tidyclust",
                                   "dplyr",
                                   "tidymodels")) %dopar% {
    # Reproducibility over parallel
    set.seed(starting_seed + i)
    result_matrix <- matrix(0, nrow = nrow(data),
```

```
                          ncol = nrow(data))


random_sample <- data |>
  filter(index %in% sample(index,
                           proportion_resample * nrow(data)))


if (algorithm == "kmeans") {
  cluster_assigned <- k_means(num_clusters = k) |>
    fit({{formula}},
        data = random_sample |>
          select(-index))
} else if (algorithm == "hier_clust") {
  cluster_assigned <- hier_clust(num_clusters = k) |>
    fit({{formula}},
        data = random_sample |>
          select(-index))
} else {
  stop("Algorithm is not supported. Please select
       one of (kmeans, hier_clust)")
}



intermediate <- data.frame(random_sample$index,
                           extract_cluster_assignment(
                             cluster_assigned
                             ) |>
                             mutate(.cluster = as.character(
```

```
                                    .cluster)
                                    ),
                                stringsAsFactors = FALSE)
    colnames(intermediate) <- c("index", "cluster")

    for (c in unique(intermediate$cluster)) {
      idx <- intermediate[intermediate$cluster == c, ]$index

      # Check that the index list is not length 1 for
      # purposes of n > m for combn
      # This also fixes a bug in the
      # previous code where idx was wrongly
      # interpreted as single numerical
      # value as param inside combn
      if (length(idx) > 1) {
        idx <- sort(idx)
        ones <- t(combn(idx, 2))
        result_matrix[cbind(ones[, 1], ones[, 2])] <- 1
      }

      neg_one_idx <- expand.grid(idx, setdiff(random_sample$index, idx))
      result_matrix[cbind(neg_one_idx[, 1], neg_one_idx[, 2])] <- -1
    }
    result_matrix[lower.tri(result_matrix, diag = TRUE)] <- NA
    result_matrix
  }
return(results)
```

```
}
```

Appendix B

MEAN MATRIX

```r
mean_matrix <- function(list_of_matrices = list_of_matrices) {
  # Absolute value the entire matrix, for all matrices
  absolute_sum <- lapply(list_of_matrices, abs)

  # Get the sum of absolute values in the final matrix
  absolute_final <- absolute_sum |>
    reduce(`+`)

  # Get the sum of values in the final matrix
  numerator <- list_of_matrices |>
    reduce(`+`)

  # Returns the matrix of zero-removed means.
  # The zero-removed mean is the sum over
  # the (length - the number of zeroes).
  # The length - the number of zeroes is
  # equivalent to the sum over absolute values
  return(numerator / absolute_final)
}
```

Appendix C

METRIC CALCULATION

```r
squared_distance_from_one <- function(mean_matrix = mean_matrix) {
  # Ignore the order of the elements in matrix
  res_vec <- as.vector(mean_matrix)

  # Absolute value and remove NAs
  res_vec <- abs(res_vec[!is.na(res_vec)])

  # Take average squared distance from one
  return(mean((1 - res_vec)^2))
}
```

Appendix D

POINT CONTRIBUTION FUNCTION

```
point_contribution <- function(data, mean_matrix) {
  # Given the original data (the dataset inputted into the resample_matrices funct
  # and the mean_matrix, returns each observation's contribution to the CRAB Score
  # We don't necessarily require the second arg to be the mean matrix, for example
  # it could be resample matrix j.
  # This is returned as an (index, score) dataframe.

  # Get a list of lists containing index and index "CRAB" score
  scores <- lapply(c(1:nrow(data)), function(i) {
    full_vec <- getPointVecHelper(i, mean_matrix)
    singlePointScore <- mean((1 - full_vec)^2)
    return(list(index = i, singlePointScore = singlePointScore))
  })
  final_df <- do.call(rbind.data.frame, scores)
  total_score <- mean(final_df$singlePointScore)

  final_df <- final_df |>
    mutate(singlePointContribution = singlePointScore / total_score)

  return(final_df)
}
```

Appendix E

GET POINT VECTOR HELPER

```
getPointVecHelper <- function(i, mean_matrix) {
  # Given a point i and the mean matrix, should
  # return vector of all values in the mean matrix corresponding to obs i
  # after removing all NAs and then taking absolute value

  row <- mean_matrix[i, ]
  col <- mean_matrix[, i]

  # We don't care about the order, just the values themselves
  res_vec <- c(row, col)
  res_vec <- abs(res_vec[!is.na(res_vec)])
  return(res_vec)

}
```

Appendix F

CRAB SCORE ACROSS K CLUSTERS

```
crab_k_across <- function(data = data,
                          c_vec = c_vec,
                          formula = ~ .,
                          number_of_resamples = 80,
                          proportion_resample = 0.8,
                          starting_seed = 599,
                          algorithm = "kmeans") {
  # Calls the resampling function across the dataset, with the parameters
  # passed to the resampling function.

  # Returns a dataframe with k rows (one for each element in the c_vec cluster vec
  # and a value for the cRab score after running the cRab algorithm on the dataset

  result <- lapply(c(c_vec), function(k) {
    resample_matrices <- resample_function(
      data = data,
      formula = formula,
      k = k,
      number_of_resamples = number_of_resamples,
      proportion_resample = proportion_resample,
      starting_seed = starting_seed,
      algorithm = algorithm
```

```
      )

    m_matrix <- mean_matrix(resample_matrices)
    score <- squared_distance_from_one(m_matrix)

    return(list(k = k,
                cRab = score))
  })
  return(dplyr::bind_rows(result))
}
```

```
# resample_matrices <- resample_function(penguins,
#                                        number_of_resamples = 40,
#                                        proportion_resample = 0.8,
#                                        starting_seed = 100)
# m_matrix <- mean_matrix(resample_matrices)
#
#
# d <- point_contribution(penguins, m_matrix)
#
# test <- lapply(resample_matrices,
#        point_contribution,
#        data = penguins)
#
# mean_var_single <- do.call(rbind.data.frame, test) |>
#   group_by(index) |>
#   summarize(mean = mean(singlePointScore),
```

```
#             var = sd(singlePointScore)) |>
#   arrange(desc(var))
```